Outline

mini-BLAST:
A BLAST-like Tool for Multiple Local Alignment

Makayla McCreary, Neha Thakar, Vedika Sridhar, and Sriya Veeramachaneni
Carnegie Mellon University
02-604: Fundamentals of Bioinformatics
Philip Compeau
April 13, 2026

General Outline:

1. Introduction
    - 1.1. Biological Background of Sequence Alignment
    - 1.2. Naive Approaches to Sequence Alignment
        - 1.2.1. Limitations of Naive Approaches
    - 1.3. Rapid sequence alignment at scale
        - 1.3.1. Why heuristics are needed vs. exact DP
        - 1.3.2. BLAST as the field standard
2. mini-BLAST Computational Pipeline Overview
    - 2.1. Preview design and Streamlit app, briefly explain that we will demonstrate on a real biological application at the end?
    - 2.2. Figure: pipeline diagram/flow chart?
3. mini-BLAST Architecture
    - 3.1. HSP seeds generation
    - 3.2. 2-hit Filtering
    - 3.3. Extension (ungapped -> s1 -> gapped -> backtracking)
        - 3.3.1. Figure for X-drop to show pruning
        - 3.3.2. Explain max bound decision
    - 3.4. Alignment Scoring and Statistics (raw score)
    - 3.5. Address params and chosen defaults?
4. User Interface
    - 4.1. Explain features/workflow with screenshots for UI design
5. Application Case: H1N1
    - 5.1. Why we are testing on H1N1 specifically
    - 5.2. Database curation with Entrez
    - 5.3. Explanation of Results
        - 5.3.1. Validation against real BLAST
6. Discussion/Conclusion
7. Limitations
    - 7.1. Heavily dependent on chosen params
    - 7.2. E-value is not as informative with a smaller database
8. Future Work
    - 8.1. PSI-BLAST for positions specific scoring params
    - 8.2. Parallelization?

9. Supplemental

9.1. Testing but not necessary

Paper

mini-BLAST:
A BLAST-like Tool for Multiple Local Alignment

Makayla McCreary, Neha Thakar, Vedika Sridhar, and Sriya Veeramachaneni
Carnegie Mellon University
02-604: Fundamentals of Bioinformatics
Philip Compeau
April 13, 2026

1. Introduction/Background/Motivation

1.1 Biological Background of Sequence Alignment

Sequence alignment is the process of arranging RNA, DNA, or protein sequences so that homologous residues are placed in the same positions, allowing researchers to infer conserved function, structural similarity, or shared ancestry. In practice, alignments are often used to identify genes and proteins, detect mutations, annotate newly sequenced molecules, and compare unknown sequences against databases. For nucleic acids, biologically meaningful similarity can reflect conserved coding regions, regulatory elements, or evolutionary descent. For proteins, amino acid conservation often preserves biochemical function even when the underlying DNA has diverged substantially. Because mutations accumulate through substitutions, insertions, and deletions, alignment methods must account for mismatches and gaps rather than requiring exact string matches.

1.2 Naive Approaches to Sequence Alignment

A simple way to compare biological sequences is to align them position by position and simply count how many symbols match. The problem with this comparison is that it assumes the correct alignment is already known, which prevents symbols in one sequence from aligning to different positions in the other. For biological sequences, this is unrealistic because insertions and deletions shift corresponding symbols so they are no longer properly aligned. Because of this, a more general notion of alignment is necessary, in which gap symbols may be inserted so that symbols in different raw positions can still be treated as corresponding.

Another naive approach is to score an alignment solely by the number of matched symbols, which seeks alignments with as many matches as possible. While this algorithm is more flexible, it still uses an oversimplified scoring model that rewards matches without adequately penalizing biologically implausible gaps between matches.

Exact methods improve on these naive comparisons. Global alignment attempts to align two sequences across their full lengths, whereas local alignment searches only for the highest-scoring matching subsequences. These approaches better reflect the challenges associated with the alignment of biological sequences.

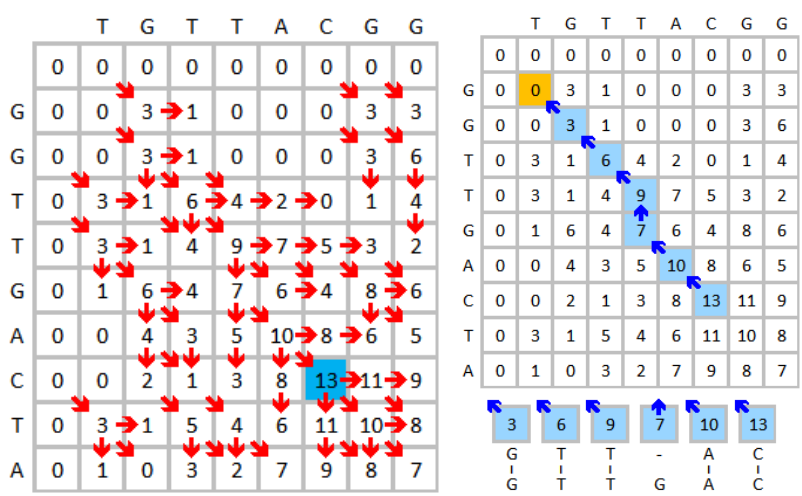


Figure 1. Classic dynamic programming approach for local alignment using the Smith-Waterman algorithm. The left panel shows a filled scoring matrix with the highest scoring cell highlighted in blue. This corresponds to the end of the local alignment. The right panel shows the backtracking process using pointers to reconstruct the final aligned strings. Images taken from Wikipedia.

1.2.1 Limitations of Naive Approaches

A primary limitation of these naive alignment approaches is computational cost. Both methods require dynamic programming, which fills in a scoring matrix whose size is proportional to the product of the two

sequence lengths. As a result, every comparison requires evaluating many possible alignments, even though most regions are unrelated. While this is manageable for a single pair of sequences, it becomes increasingly slow when one query must be searched against a large database, which is why heuristic methods such as BLAST are needed.

1.3 Rapid Sequence Alignment at Scale

In order to provide a practical platform for sequence alignment against large databases, exact approaches such as Smith-Waterman are not efficient, motivating the development of the BLAST heuristic. The underlying logic revolves around the observation that there are relatively few high-scoring character matches distributed across a typical dynamic programming matrix, and from those, only a fraction have the potential for good results. This observation gave way to the seed-and-extend approach BLAST uses for vastly improving the speed and scalability of local alignment. It has become a popular tool because of this fact and due to its easy accessibility. NCBI BLAST is regarded as a gold standard implementation of the algorithm, and is available in both web-based and downloadable code-based platforms. Variations of BLAST focused on specific optimizations commonly refer to the NCBI tool to validate their output accuracy.

2. mini-BLAST Computational Pipeline

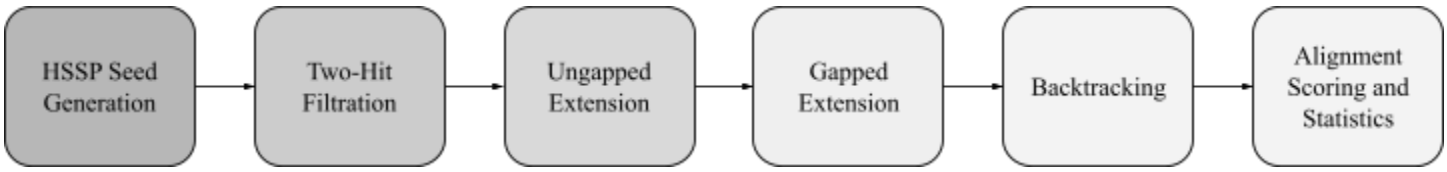


Figure 2. Overview of the mini-BLAST computational pipeline. Each stage aims to reduce the final number of full gapped alignment that must be performed using some filter/thresholding.

The program takes the following inputs:

- FASTA file or custom query sequence
- Database with reference sequences
- The ungapped extension threshold (S1)
- The two-hit diagonal window (A)
- A number n that designates the intervals at which different references are selected

These inputs are fed into the engine which aligns the query sequence to the selected database references. The output contains the alignment results which include:

- Reference index and ID
- Raw alignment score
- Bit score
- E-value
- Query coverage
- Percent Identity
- Position

The algorithm's process and its inputs and outputs are discussed in more detail in subsequent sections.

3. Methods

3.1 Seed Generation

To reduce the search space and overall runtime of alignment, high scoring segment pairs (HSSPs) are identified between the query sequence and each reference. These HSSPs are ungapped alignments that meet or exceed a defined scoring threshold (default: allowing one mismatch) between kmers in the query and the reference which serve as seeds for extension. Potential HSSPs are derived from each query kmer using a branch and bound approach and the resulting list contains all kmers that would meet or exceed the scoring threshold if identified in the reference sequence. Thus, we search for each potential HSSP within the reference. To improve lookup speed, each reference kmer is numerically encoded using the below formula and stored in a hash table, where the table index represents the kmer position in the reference sequence. Query kmers are converted to numerical encodings during comparison. For each HSSP that is identified, its position in both the query and reference sequences are stored, as well as the score of their ungapped alignment. These HSSPs will be further filtered using the two-hit filtration method discussed below before implementing extension.

$$v \times 4^{k-p-1}$$

Nucleotide Encoding

	A	C	G	T
v	0	1	2	3

Amino Acid Encoding

	A	C	D	E	F	G	H	I	K	L	M	N	P	Q	R	S	T	V	W	Y
v	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

Equation 1: Numerical encoding of kmers is based on their character value, v, as indicated in the above tables; the kmer length, k; and the position of each character in the kmer, p.

Figure 3. BLAST uses short k-mers to generate seeds instead of comparing every position in the query and reference sequence to each other. Image taken from Lobo (2008).

2.2 Two-Hit Filtration

Two-hit filtering is a heuristic used to reduce the number of seeds that proceed to the expensive extension phase, without sacrificing sensitivity. The central idea is that a genuine local alignment is unlikely to arise from a single short k-mer match by chance, but two nearby k-mer matches on the same diagonal are much less likely to be coincidental. To preserve sensitivity, the score threshold T for declaring a word hit is lowered, so the algorithm finds more raw hits overall, but only a small fraction satisfies the two-hit criteria and proceeds further. Altschul et al. showed that this tradeoff improves efficiency while maintaining, and for sufficiently

strong HSPs even improving, sensitivity relative to the original one-hit heuristic. By requiring two nearby hits, BLAST sharply reduces the number of costly extensions it performs.

After seed generation, each seed is characterized by its diagonal, defined as the difference between its start position in the query and its start position in the reference. Seeds that fall on the same diagonal represent matches that are in the same frame with no indels between them. The filter then scans each diagonal independently, sorting hits by query position. For two consecutive hits on the same diagonal to form a valid pair, they must satisfy two conditions: they must be non-overlapping, and they must be within the diagonal window A of each other. When a valid pair is found, the second hit is passed to extension.

2.3 Extension

The extension phase of the mini-BLAST architecture represents the critical transition from rapid k-mer matching and filtering to biologically meaningful local alignments. The pipeline is initiated after the 2-hit filtration stage, which identifies pairs of seeds on the same diagonal. The goal of the extension pipeline is to verify if these initial hits are part of a promising alignment, while minimizing the computational burden of a full-scale dynamic programming approach. The process follows a tiered hierarchy, where each stage aims to further reduce the number of computationally intensive gapped alignments that are performed at the end.

2.3.1 The X-drop Heuristic and Search Space Banding

In classic sequence alignment, the search space is often limited by a band of some Manhattan distance *d* from the main alignment diagonal passing through a seed. This fixed banding reduces space allocation from O(NM) for two sequences of lengths N and M to $O(d \cdot \min(M, N))$. While fixed banding works well for highly similar sequences where the optimal alignment path stays relatively close to the diagonal, it often fails for dissimilar sequences where gaps force the alignment to significantly deviate from the diagonal.

Gapped BLAST replaces this manual, fixed banding with an adaptive banding approach using the X-drop criterion. Rather than using a rigid corridor to limit the search space, X-drop allows the algorithm to explore only cells for which the score has not dropped more than X below the best score found so far in that particular alignment. This gives a dynamic search “cloud” that expands and contracts based on the quality of the match, allowing the algorithm to follow a promising alignment even when it veers from the main diagonal.

However, even with X-drop pruning, there are some cases of sequence compositions where the pruning mechanism may fail to terminate an alignment extension early enough. For example, if both the query and reference sequences contain long stretches of repetitive sequences (low complexity regions), the extension algorithm may find enough matches to keep the score from dropping significantly. The score may plateau instead of dropping, and X-drop will fail to terminate the extension. Additionally, if the gap extension penalty is low relative to the X-drop threshold, the algorithm will explore many possible insertions and deletions – since the cost of gapping is cheap, the alignment doesn’t lose enough points to trigger pruning. In such cases, it is useful to set a hard *max_band* limit in addition to X-drop pruning.

In our mini-BLAST implementation, we have tested two *max_band* settings: scoring-based and length-based bound. The scoring-based bound is set to $2 \times (X / \min(\text{match}, \text{mismatch}))$, and is derived from X-drop’s mathematical limit. It calculates the maximum distance an alignment path could travel before X-drop must trigger. Length-based bounding sets the limit based on the length of the query sequence. We found these settings demonstrated a tradeoff between speed and sensitivity; length-based bounding results in higher computational costs since the “cloud” is able to expand further from the seed, but often resulting is higher quality alignments. On the other hand, scoring-based bounding was comparatively much faster due to stricter banding, but often gave truncated alignments. This is because biological alignments usually have regions of low

conservation before rebounding to high-scoring regions, and a more conservative bound cuts off alignments before they have a chance to recover. As such, we have decided to use the length-based bounding approach as a hard limit and safety check in the case the X-drop pruning fails.

2.3.2 Ungapped Extension and S_1 Threshold

Starting from input coordinates of the original seed pairs in the query and reference sequences, mini-BLAST first performs a greedy extension centered around the initial seed in both the forward and backward directions. In this stage, we ignore insertions and deletions and focus solely on substitutions, resulting in a cumulative score based on match/mismatch parameters or a substitution matrix like BLOSUM62. To prevent the algorithm from unnecessarily traversing the entire length of the sequence, the X-drop pruning approach is used: extension continuous until the current score S falls more than X_{ungap} below the maximum score S_{max} achieved during that specific extension. The resulting ungapped score is compared against a pre-defined S_1 threshold, any only those that pass meet or exceed this value continue to gapped extension.

In real BLAST, the S_1 threshold is based on the expected value (E-value) and acts as a statistical filter – if an ungapped alignment cannot reach a score of S_1, it is unlikely to contribute to a significant gapped alignment and is therefore discarded. However, the statistical power of the E-value is reduced in mini-BLAST due to the significantly smaller reference database. In a smaller search space, a score that might be highly significant (having a lower E-value) can easily be achieved by random chance. Conversely, a meaningful biological signal may be discarded because the background noise is too high to establish a rigorous threshold. The current S_1 implementation instead acts as a simple "heuristic filter" rather than a statistical one. Since we are not able to leverage the background distribution of an extremely large database, the S_1 value must be manually set based on the expected length and conservation of the sequences being aligned. If S_1 is set too high, the algorithm risks missing true positives that have lower ungapped scores. If it is set too low, we risk triggering gapped extension for many low-quality hits, undoing the speed advantages of the many heuristics used in BLAST.

2.3.3 Gapped Extension

For surviving candidates, we then transition to a gapped extension using a 3-state affine gap penalty implementation with M (match/mismatch), I (insertion/gap in reference), and D (deletion/gap in query) states. Unlike standard Smith-Waterman implementations that compute a full N x M matrix, mini-BLAST uses a seed-centered, "cloud"-based approach with the following recurrence:

$$M_{i,j} = \max(M_{i-1,j-1} + s(q_i, r_j), I_{i,j}, D_{i,j})$$
$$I_{i,j} = \max(M_{i,j-1} - \text{gapOpen}, I_{i,j-1} - \text{gapExtend})$$
$$D_{i,j} = \max(M_{i-1,j} - \text{gapOpen}, D_{i-1,j} - \text{gapExtend})$$

This recurrence is computed outward in layers from the seed, where the "cloud" of explored cells is dynamically pruned using the gapped X-drop parameter X_g. By only calculating cells (i, j) where the state scores remain within X_g of the best score found in the current extension, the algorithm "follows" the path of biological significance rather than wasting cycles on low-scoring regions of the alignment matrix.

Rather than allocating a static N x M array, which consumes O(NM) space regardless of X-drop pruning, the algorithm instead stores state scores and backtracking pointers in dictionaries where keys are (i, j) coordinates relative to the seed. If the X-drop logic determines a cell is not worth exploring, the coordinate is never instantiated in the dictionary. This approach gives a "matrix" where the memory consumed is directly

proportional only to the cells that are actually visited during the alignment, rather than the theoretical maximum search space spanning the full lengths of both the query and reference sequences.

2.3.4 Final Backtracking and Validation

To enable backtracking to reconstruct the final aligned sequences, a dictionary *back* stores a tuple containing the previous coordinate and the state (M, I, or D) that led to the current maximum during the gapped extension phase. This gives efficient O(L) reconstruction of the aligned sequences, where L is the length of the alignment. During the backtracking phase, the algorithm traverses pointers from the *best_pos* back towards the seed. A final check, X_{final}, is also applied in this stage as a cleanup mechanism, enabling us to remove any low-scoring "tails" from the alignment. This ensures that the final reported alignment is not only optimal in score, but also more biologically meaningful as it is free of any trailing gap-heavy regions that do not contribute significantly to sequence homology.

2.4 Alignment Scoring and Statistics

2.4.1 Query Coverage

Query coverage indicates the percentage of the query that was aligned against a given reference. This is calculated as the length of the aligned query over the length of the original query multiplied by 100. Not all seed extensions will continue for the full length of a query, so this measure can provide useful information regarding conserved gene identity or function, or conserved regions of a larger query sequence that may indicate important motifs or divergent gene evolutions.

2.4.2 Percent Identity

Percent identity measures the similarity between two aligned sequences as the proportion of alignment positions that contain the same character. After gapped alignment, the aligned query and reference strings are compared position by position, including any gap characters introduced by insertion or deletions. A position is counted as identical only when both sequences contain the same non-gap character. The number of identical positions is then divided by the total alignment length, including gap columns, and multiplied by 100. Because the denominator is the full alignment length, gaps reduce the percent identity score rather than being ignored.

2.4.3 Bit Score

Once the algorithm finishes, it returns alignments with raw scores. Because raw scores depend on scoring parameters and sequence length, they cannot be directly compared across searches (Altschul, n.d.). Therefore, the algorithm normalizes raw scores into bit scores using Equation 2, which takes the raw score S along with the statistical parameters λ and K.

$$S' = \lambda * S - \frac{\ln K}{\ln 2}$$

Equation 2 (Karlin & Altschul, 1990): Bit score calculations

2.4.4 E-value

After the bit score is calculated, it serves as input to the E-value calculation shown in Equation 3. The E-value is needed to determine whether a given bit score could have been achieved by chance alone. Karlin and Altschul modeled this using an extreme value distribution to estimate the number of alignments with an equal or better bit score expected by chance in a database of the same size (Karlin &

Altschul, 1990). Unlike raw or bit scores, E-values are directly comparable across searches with different query lengths and database sizes, making them the primary metric for evaluating the biological significance of BLAST results.

$$E = m * n * 2^{-S'}$$

Equation 3 (Altschul, n.d.): E-Value

2.5 Explanation of Selected Parameter and Default Values

Throughout the program, the algorithm uses a scoring matrix to evaluate alignments during the extension phase. At every position in the alignment, the algorithm assigns a score based on the following criteria: matches are positions with the same nucleotide, mismatches are positions with different nucleotides, and gaps represent insertions or deletions. Affine gap penalties are used, in which opening a gap incurs a larger penalty than extending one. This reflects the biological reality that a single insertion or deletion event typically produces a contiguous run of gaps, making it more likely that consecutive gap positions belong to the same evolutionary event than that multiple independent events occurred. For miniBLASTn, the match reward is 2, the mismatch penalty is -3, the gap opening penalty is 5, and the gap extension penalty is 2.

The seeding phase of the algorithm uses the k-mer size parameter to determine the initial window length over which to apply the scoring matrix. mini-BLASTn uses a default k-mer size of 7, which ensures that there is a (1/4)^7 likelihood of a random 7-mer perfectly matching a query k-mer by chance, making seeds statistically meaningful. The two-hit requirement states that candidate seeds must be within a specified distance A of one another on the diagonal before moving to extension. The default distance is set to 40 and was empirically validated by Altschul et al. (1997) as the optimal balance between sensitivity and computational efficiency.

The extension part of the algorithm works in three successive phases: ungapped extension, gapped extension, and final extension. Each extension terminates when the current score drops x-below the alignment's max value. The x-drop ungapped value is set to 20 as default. The ungapped extension also gets another parameter called S1, which decides whether the final score of the ungapped extension is good enough to move onto the gapped extension phase. Next, the gapped extension takes place with an x-drop gapped value of 30. Lastly, the x-drop final value is set to 100, providing a more lenient termination criterion that recovers any alignment endpoints that may have been clipped during gapped extension.

The λ and K parameter values are used to normalize the raw score after the final gapped extension. These values were derived from the scoring matrix parameters using Karlin-Altschul statistics (Karlin & Altschul, 1990). MiniBLAST uses BLASTN default gap open and extension penalties of 5 and 2, with corresponding fixed values of $\lambda = 0.625$ and $K = 0.41$.

2.6 Database Construction and Output Validation

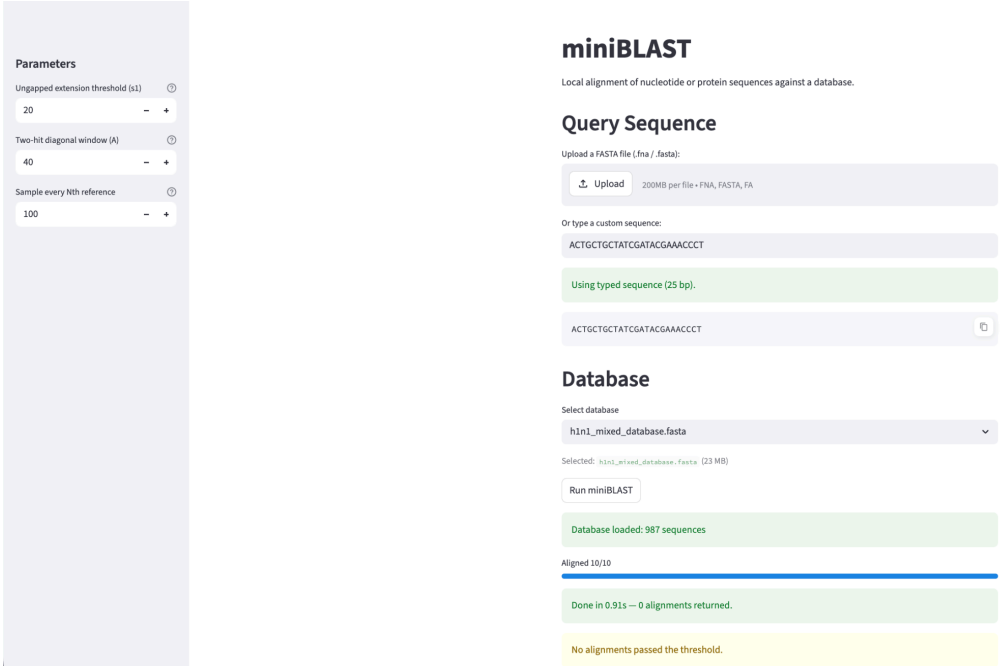
The standard non-redundant nucleotide and protein databases used by NCBI contain roughly 96 million and 595 million sequences respectively. Given the scale and computational resources available for this project, custom databases were curated using BioPython's Entrez package, which allows users to fetch sequences out of NCBI's database. The specific composition of the databases generated thus far are described in the "Applications" subsections. Each is intended to cover a range of highly similar, somewhat similar, and background sequences in order to provide meaningful results.

In order to validate the results of mini-BLAST, queries were run through BLAST+, a command-line NCBI tool that provided the flexibility to use our own custom databases rather than pre-existing NCBI

databases, a functionality which is unavailable on their web-based BLAST platform. While we are unable to specify all hyperparameters implemented in mini-BLAST through this method, BLAST+ provides a closer to one-to-one comparison between mini-BLAST outputs and established BLAST software.

4. User Interface

The miniBLAST application was built using Python's Streamlit package. After entering `streamlit run app.py` into the command terminal, a web-based graphical user interface opens. Users can enter a custom query sequence via the text box or upload a FASTA file. Next, users select the database they would like to align the query against. Editable parameters include S1 and A, with defaults of 20 and 40 respectively. Users can also choose to align against every nth reference in the database, with the default set to 100. Once all required fields are filled, users can run miniBLAST. Successful alignments are displayed in a results tab with a downloadable CSV file. The best alignment is defined as the query-reference pair with the lowest E-value. Alignments that do not pass the score threshold are not reported, and if no alignments succeed, users receive a "no alignments passed the threshold" message.



5. Applications

5.1 mini-BLASTn: H1N1

H1N1 is a well documented virus with a small genome, providing a good source of validation-ready data. Additionally, the 2009 H1N1 virus was a result of several reassortments of strains from swine, avian, and human populations, making it an interesting subject for sequence similarity comparison. The mini-BLAST database curated for H1N1 nucleotide sequence alignment contains 987 reference sequences from H1N1, H3N2, H5N1, SARS-CoV-2, rhinovirus, and 16S bacterial rRNA.

Results… currently working on interpretation. Alignment of H1N1 segment 1 aligns with other H1N1 segment 1 references with the highest query coverage. Originally, this value was only around 1.3% due to the short length of all alignments returned. While this still is useful for determining relative similarity of query to

each reference, longer alignments would provide greater confidence. After some investigation, changing the max band produced more accurate results (97% query coverage, 87% identity), with the drawback of much slower runtime. To try and improve this, we would like to look into the programs DUST and SEG, which NCBI BLAST uses to mask low complexity regions, which are regions of low diversity and high false positive rates.

6. Discussion

6.1 Limitations

Runtime - in favor of greater output accuracy, seed extension runtime is much slower than would be practical for large scale use, taking roughly 1 hour to align an H1N1 query against 10 reference sequences. This could be improved by incorporating programs like DUST or SEG to mask low complexity regions, or by parallelizing the code to align against multiple references simultaneously.

Database size - compared to NCBI BLAST, the number of sequences in our databases is significantly limited and the composition more narrow. This can disrupt statistical significance of calculated E-values, though attempts to compensate were made by ensuring to include background sequences for noise. It may also prevent identification of relevant or novel alignments. That said, other versions of BLAST, such as CAAD BLAST, have intentionally implemented database prefiltering steps as a method of improve runtime while maintaining accuracy.

Handling IUPAC ambiguous nucleotide codes - sequence assemblies often have ambiguous nucleotides representing low-confidence sequence positions. mini-BLAST currently ignores these codes in scoring during the seed generation and does not incorporate them during seed extension.

6.2 Future Directions

- DUST for masking low-complexity regions
- Position-specific scoring (PSI-BLAST)
- Allowing for different scoring matrices .. fairly easy to do technically but comparison of alignment results from different scoring matrices requires effort

Citations

References

Karlin, S., & Altschul, S. F. (1990). Methods for assessing the statistical significance of molecular sequence features by using general scoring schemes. Proceedings of the National Academy of Sciences of the United States of America, 87(6), 2264–2268. https://doi.org/10.1073/pnas.87.6.2264

Altschul, S. F. (n.d.). *The statistics of sequence similarity scores*. National Center for Biotechnology Information. https://www.ncbi.nlm.nih.gov/BLAST/tutorial/Altschul-1.html#head3

https://www.ncbi.nlm.nih.gov/books/NBK279684/
https://www.nlm.nih.gov/ncbi/workshops/2023-08_BLAST_evol/databases.html#:~:text=The%20very%20large%20%E2%80%94%2096%20million,people%20who%20use%20BLAST%20locally

Intro
https://www.ncbi.nlm.nih.gov/books/NBK464187

Two hit
https://academic.oup.com/nar/article/25/17/3389/1061651#19090142

Database
https://www.nlm.nih.gov/ncbi/workshops/2023-08_BLAST_evol/databases.html#:~:text=The%20very%20large%20%E2%80%94%2096%20million,people%20who%20use%20BLAST%20locally.

Rapid Alignment at Scale Ref
https://www.bu.edu/caadlab/ICS_2010.pdf
BLAST FAQ
https://blast.ncbi.nlm.nih.gov/doc/blast-help/FAQ.html
H1N1 Bio Background
https://pmc.ncbi.nlm.nih.gov/articles/PMC4910438/